Data Structures (ETCCDS202)

Unit–2 Assignment: Linear Data Structures

Course Details

Name of the School:

School of Engineering & Technology

Program -CSE Robotics and AI

Course Title: Data Structures

Course Code: ETCCDS202

Unit Number:2

Unit Title: Linear Data Structures

Student Name:Ajitesh Singh Gusain

Section:A

Submission Date:19|3|26

Roll No.-2501630014

Task 1: Dynamic Array Simulation (Like Python List)

<u>Goal: To understand and simulate the resizing logic of a dynamic array, similar to how a Python list handles memory allocation.</u>

Implementation Logic:

- Capacity Management: The array starts with a small initial capacity (e.g., 2).
- Resizing: When the number of elements matches the current capacity, a new internal array is created with double the previous capacity.
- Element Transfer: All existing elements are copied from the old array to the new, larger array before the new element is appended.

Amortized Time Complexity of append():

- Worst-Case: When the array is full, the append() operation triggers a resize, which requires copying n elements. This specific operation is O(n).
- Average Case: Most append() operations simply involve inserting an element into an empty slot, which is O(1).
- Conclusion: Because the doubling of capacity happens infrequently, the "expensive" O(n)operations are spread out over many O(1) operations. Therefore, the amortized time complexity for append() is O(1).

lst_toolkit.py × output.txt

```python
class DynamicArray:
    def __init__(self, capacity=2):
        self.n = 0  # Number of actual elements
        self.capacity = capacity # Max slots available [cite: 62]
        # We create a list of fixed size filled with None
        self.A = [None] * self.capacity

    def append(self, x):
        # If array is full, create a new array of double capacity [cite: 64]
        if self.n == self.capacity:
            self._resize(2 * self.capacity)

        self.A[self.n] = x # Insert new element at the next available slot [cite: 66]
        self.n += 1

    def _resize(self, new_cap):
        print(f"--- Resizing: {self.capacity} -> {new_cap} ---")
        # Create a new "plain list" for the larger capacity [cite: 51, 65]
        B = [None] * new_cap

        # Copy elements to new array manually [cite: 65]
        for k in range(self.n):
            B[k] = self.A[k]

        self.A = B # Replace old array with the new one
        self.capacity = new_cap

    def pop(self):
        # Remove last element and return it [cite: 67]
        if self.n == 0:
            return "Array is empty"
        val = self.A[self.n - 1]
        self.A[self.n - 1] = None # Clear the slot
        self.n -= 1
        return val

    def __str__(self):
        # Show only the elements currently in the array [cite: 68]
        return "[" + ", ".join(str(self.A[i]) for i in range(self.n)) + "]"
```

-
-

- ```
  ldst_toolkit.py    output.txt    Code
  PS C:\Users\ajite\OneDrive\Desktop\ASSIGNMENT FOLDER\assignment of second sem\dsa assignment\Assignment_Ajitesh_singh_gusai
  rive\Desktop\ASSIGNMENT FOLDER\assignment of second sem\dsa assignment\Assignment_Ajitesh_singh_gusain_2501630014_LAB_unit-
  Initial Capacity: 2
  Appended 1: Current Array [1]
  Appended 2: Current Array [1, 2]
  --- Resizing: 2 -> 4 ---
  Triggered Resize: Appended 3, Capacity grew to 4
  Appended 4: Current Array [1, 2, 3, 4]
  --- Resizing: 4 -> 8 ---
  Triggered Resize: Appended 5, Capacity grew to 8
  Appended 6: Current Array [1, 2, 3, 4, 5, 6]
  Appended 7: Current Array [1, 2, 3, 4, 5, 6, 7]
  Appended 8: Current Array [1, 2, 3, 4, 5, 6, 7, 8]
  --- Resizing: 8 -> 16 ---
  Triggered Resize: Appended 9, Capacity grew to 16
  Appended 10: Current Array [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

  --- Performing 3 Pops ---
  Popped: 10 | Current Array: [1, 2, 3, 4, 5, 6, 7, 8, 9]
  Popped: 9 | Current Array: [1, 2, 3, 4, 5, 6, 7, 8]
  Popped: 8 | Current Array: [1, 2, 3, 4, 5, 6, 7]
  Final State: [1, 2, 3, 4, 5, 6, 7] (Size: 7, Capacity: 16)
  PS C:\Users\ajite\OneDrive\Desktop\ASSIGNMENT FOLDER\assignment of second sem\dsa assignment\Assignment_Ajitesh_singh_gusai
  ```

Task 2: Linked List Operations

Advantage: Unlike arrays, linked lists do not require shifting elements during insertion or deletion, making these operations O(1) if the pointer is already at the location.

Doubly vs Singly: The Doubly Linked List allows for bidirectional traversal and easier deletion of a node if you only have a reference to the node itself.
  ```

```python
#tASK-2
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
        self.prev = None  # Used for Doubly Linked List

class SinglyLinkedList:
    def __init__(self):
        self.head = None

    def insert_at_beginning(self, x):
        new_node = Node(x)
        new_node.next = self.head
        self.head = new_node

    def insert_at_end(self, x):
        new_node = Node(x)
        if not self.head:
            self.head = new_node
            return
        curr = self.head
        while curr.next:
            curr = curr.next
        curr.next = new_node

    def delete_by_value(self, x):
        curr = self.head
        if curr and curr.data == x:
            self.head = curr.next
            return
        prev = None
        while curr and curr.data != x:
            prev = curr
            curr = curr.next
        if curr:
            prev.next = curr.next

    def traverse(self):
        nodes = []
        curr = self.head
        while curr:
            nodes.append(str(curr.data))
            curr = curr.next
        print(" -> ".join(nodes) if nodes else "Empty List")

class DoublyLinkedList(SinglyLinkedList):
    def insert_after_node(self, target, x):
        curr = self.head
        while curr and curr.data != target:
            curr = curr.next
        if curr:
            new_node = Node(x)
            new_node.next = curr.next
            new_node.prev = curr
            if curr.next:
                curr.next.prev = new_node
            curr.next = new_node

    def delete_at_position(self, pos):
        # Using 0-based indexing as requested
        if not self.head: return
        curr = self.head
        if pos == 0:
            self.head = curr.next
            if self.head: self.head.prev = None
            return
        for _ in range(pos):
            if curr.next: curr = curr.next
        if curr.prev: curr.prev.next = curr.next
        if curr.next: curr.next.prev = curr.prev
```

```
R\assignment of second sem\dsa assignment\Assignment_Ajitesh_singh_gusain_2501630014_LAB_unit-2\ldst_t
--- Task 2(A): Singly Linked List ---
After insertions:
30 -> 20 -> 10 -> 40 -> 50 -> 60
After deleting 20:
30 -> 10 -> 40 -> 50 -> 60

--- Task 2(B): Doubly Linked List ---
After inserting 99 after 2:
1 -> 2 -> 99 -> 3
After deleting at position 1:
1 -> 2 -> 99 -> 3
PS C:\Users\ajite\OneDrive\Desktop\ASSIGNMENT FOLDER\assignment of second sem\dsa assignment\Assignmer
```

Task 3: Stack & Queue From Scratch (Using Singly Linked List)

To ensure O(1) efficiency for the Stack, we perform all operations at the **head**. For the Queue, we maintain a **tail pointer** so that adding to the back is also O(1).

Operation	Time Complexity	Explanation
Stack Push/Pop	O(1)	Performed at the head of the linked list; no traversal needed.
Queue Enqueue	O(1)	Achieved by maintaining a tail pointer to the end of the list.
Queue Dequeue	O(1)	Performed by moving the head pointer forward one

Operation	Time Complexity	Explanation
		step.

```python
#task 3
class Stack:
    """Stack ADT (LIFO) using Singly Linked List storage [cite: 94]"""
    def __init__(self):
        self.storage = SinglyLinkedList()
        self.size = 0

    def push(self, x):
        # Insert at beginning for O(1) efficiency
        self.storage.insert_at_beginning(x)
        self.size += 1

    def pop(self):
        # Remove from beginning for O(1)
        if self.is_empty():
            print("Stack Underflow!")
            return None
        val = self.storage.head.data
        self.storage.head = self.storage.head.next
        self.size -= 1
        return val

    def peek(self):
        return self.storage.head.data if self.storage.head else None

    def is_empty(self):
        return self.size == 0

class Queue:
    """Queue ADT (FIFO) using Singly Linked List with Tail Pointer [cite: 100, 105]"""
    def __init__(self):
        self.head = None
        self.tail = None
        self.size = 0

    def enqueue(self, x):
        # Add to tail for O(1)
        new_node = Node(x)
        if not self.tail:
            self.head = self.tail = new_node
        else:
            self.tail.next = new_node
            self.tail = new_node
        self.size += 1

    def dequeue(self):
        # Remove from head for O(1)
        if self.is_empty():
            print("Queue Underflow!")
            return None
        val = self.head.data
        self.head = self.head.next
        if not self.head:
            self.tail = None
        self.size -= 1
        return val

    def front(self):
        return self.head.data if self.head else None

    def is_empty(self):
        return self.size == 0
```

```
--- Task 3(A): Stack Operations (LIFO) ---
Peek after pushing 10, 20: 20
Popped: 20
Stack empty? False

--- Task 3(B): Queue Operations (FIFO) ---
Front after enqueuing A, B: A
Dequeued: A
Queue empty? False
```

Task-4

Task 4: Application - Balanced Parentheses Checker

Goal: To apply the Stack Data Structure to solve a real-world expression validation problem.

1. Implementation Logic

- **Algorithm**: The function is_balanced(expr) iterates through each character of a string.
- **Pushing**: Whenever an opening bracket ((, {, [) is encountered, it is pushed onto the custom Stack.
- **Popping**: When a closing bracket (), },]) is encountered, the top element of the Stack is popped and compared.
- **Validation**: The expression is considered **unbalanced** if a closing bracket does not match the popped opening bracket, or if the Stack is empty when a closing bracket appears.
- **Final Check**: After processing the entire string, the Stack must be empty for the expression to be truly balanced.

2. Required Test Case Results

Input String	**Expected Result**	**Logic Applied**
"([])"	**Balanced**	[and (are pushed; then) matches (and] matches [.
"([)]"	**Not Balanced**	[and (are pushed;] is encountered but does not match the top (.
"(((("	**Not Balanced**	Three (are pushed, but the stack remains non-empty at the end.
"" (empty)	**Balanced**	No brackets to process; stack starts and ends empty.

3. Complexity Analysis

- **Time Complexity**: O(n), where n is the length of the string, as we traverse the string exactly once.
- **Space Complexity**: O(n) in the worst case (e.g., a string of all opening brackets), as they are all stored in the Stack

```python


def is_balanced(expr):
    """
    Checks if an expression has balanced (), {}, and [][cite: 122].
    Uses the custom Stack implementation from Task 3[cite: 118].
    """
    s = Stack()
    # Define matching pairs for easy lookup [cite: 122]
    matches = {')': '(', '}': '{', ']': '['}

    for char in expr:
        if char in "({[":
            # Push opening brackets onto the stack [cite: 119, 120]
            s.push(char)
        elif char in ")}]":
            # If stack is empty or top doesn't match, it's unbalanced [cite: 121]
            if s.is_empty() or s.pop() != matches[char]:
                return False

    # If the stack is empty at the end, all brackets were matched [cite: 120, 127]
    return s.is_empty()


def test_parentheses_checker():
    print("\n--- Task 4: Balanced Parentheses Checker ---")

    # Required test cases from the assignment [cite: 123]
    test_strings = {
        "([])": True,      # Balanced [cite: 124]
        "([)]": False,     # Not balanced [cite: 125]
        "(((": False,      # Not balanced [cite: 126]
        "": True           # Empty string is balanced [cite: 127]
    }

    for expr, expected in test_strings.items():
        result = is_balanced(expr)
        status = "Balanced" if result else "Not Balanced"
        print(f"Input: '{expr}' -> Result: {status} (Expected: {expected})")
```

```
--- Task 4: Balanced Parentheses Checker ---
Input: '([])' -> Result: Balanced (Expected: True)
Input: '([)]' -> Result: Not Balanced (Expected: False)
Input: '(((' -> Result: Not Balanced (Expected: False)
Input: '' -> Result: Balanced (Expected: True)
PS C:\Users\ajite\OneDrive\Desktop\ASSIGNMENT FOLDER\assignment of second sem\dsa assignment\Assignment_Ajitesh_singh_gusain_2501630014_LAB_unit-2>
```